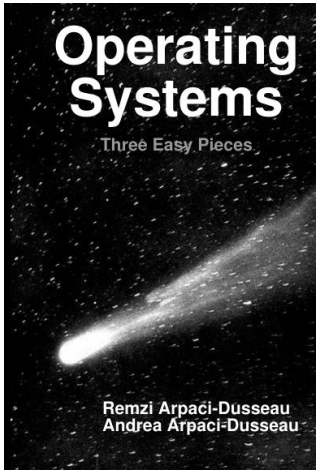
Operating Systems and Parallel Computing

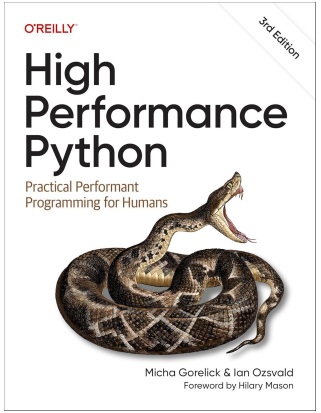
Lecture 1

Prof. Matteo Turilli

Welcome!

- Attendance
  - Automatic failure below 85% presence imposed by the department.
  - Do not assume you will never be sick, need an unplanned absence, being overwhelmed.
  - I have no power over these rules and no power to accommodate
- Blackboard:
  - All teaching material: Lectures, assignments, notebooks, etc.
  - Slides are made available after each session to allow for correcting mistakes and avoiding the 'surprise effect' of open questions :-)
- Lectures are based on:
  - Arpaci-Dusseau, Remzi H., and Andrea C. Arpaci-Dusseau. **Operating systems: Three easy pieces**. Madison, WI, USA: Arpaci-Dusseau Books, LLC, 2018. https://pages.cs.wisc.edu/~remzi/OSTEP/
  - Gorelick, Micha, and Ian Ozsvald. **High Performance Python: Practical Performant Programming for Humans**, 3rd Edition O'Reilly Media, Inc., 2025.





Planning

Assignments, exams, presentations, and grading:

Deliverable	# Session	Date	Grading
Lab report I	11	Tue 29/09/26	10%
Midterm exam	15	Wed 07/10/26	20%
Team formation	16	Tue 13/10/26	-
Lab report 2	23	Tue 10/11/26	10%
Final exam	28	Wed 25/11/26	20%
Group project	29-30	Tue 15/12/26	30%
Class participation	All	All	10%

Teaching and Cheating

- Teaching ≠ convincing students to learn
  - Sharing experience and knowledge about topics of interest
  - Not convincing about the importance of those topics for the student's goals
  - Passing an exam and graduating ≠ acquiring the instrument needed to get a job/build a career
- Cheating:
  - Outsourcing learning activities, i.e., avoiding learning
  - Examples:
    - Pretending that the work of someone/something else is our own is cheating
    - Using someone/something else work to learn and elaborate our own is not cheating
  - Who is actually cheated when we cheat?

Teaching and Cheating

- Teaching ≠ convincing students to learn
  - Sharing experience and knowledge about topics of interest
  - Not convincing about the importance of those topics for the student's goals
  - Passing an exam and graduating ≠ acquiring the instrument needed to get a job/build a career
- Cheating:
  - Outsourcing learning activities, i.e., avoiding learning
  - Examples:
    - Pretending that the work of someone/something else is our own is cheating
    - Using someone/something else work to learn and elaborate our own is not cheating
  - Who is actually cheated when we cheat? **Ourselves**
- Diversification and fairness:
  - Learning in multiple ways and venues
  - LLMs offer yet another learning venue
  - Always quote the source of the material that is not our own

The LLM in the Room

- Large language models (LLMs) are here to stay
- Learning to use them to learn:
  - Difference between asking a LLM for a solution Vs asking to help to find a solution Vs asking to teach you what is needed for YOU to find the/a solution
  - LLM as an adaptive socratic tutor
  - Adaptive = explanations depend on our current level of understanding and knowledge. It can be high for a question and very low for the next one. LLM can adapt the amount and quality of information provided based on our pre-existing proficiency.
  - **Remember: you will not be allowed to use a LLM during your job interviews!**
- Policy:
  - You can use them but you have to declare how you used them, i.e., attach our prompt(s) in a separate document. E.g., Question 1, prompts: …, etc.
  - Be careful: The Department and I do not need direct proof of their use but only a reasonable doubt.
  - Gentleman agreement: you commit to learn, I commit to make learning feasible and useful for your future career. Deal? 8-)

Group Projects

- Real learning experience:
  - You will always work in a team
  - Group dynamics, roles within a group, development methodology, shared productivity
- Accountability:
  - We use GitHub to trace each group member effort.
  - Single commit = all group failed;
  - Disparity in number and quality of the commits = individual grading;
  - Properly distributed and granular commits over time by all the members = gorup grading.
- Team formation:
  - Project and empty groups are already setup in BlackBoard
  - We have 15 sessions to form the teams, failure to create a team by session 16 = group project failed.
- Goals:
  - Choose a real problem requiring analytics workload, implement a sequential solution as a baseline, implement the fastest solution you can, measure performance on the cluster, discuss the results;
  - Adopt one technique related to this course but **not taught in it** and teach it during our presentation.
  - Two class milestones with feedback before anything is graded publicly.

Only Two Simple Behavioral Rules

1. Arrive to class on time because otherwise marking presences becomes a mess.
2. No side talking because otherwise I lose track of what I am thinking/saying.

As far as your remain civil, I do not care if you:

Censored ;)

Course Topics and Software Environment

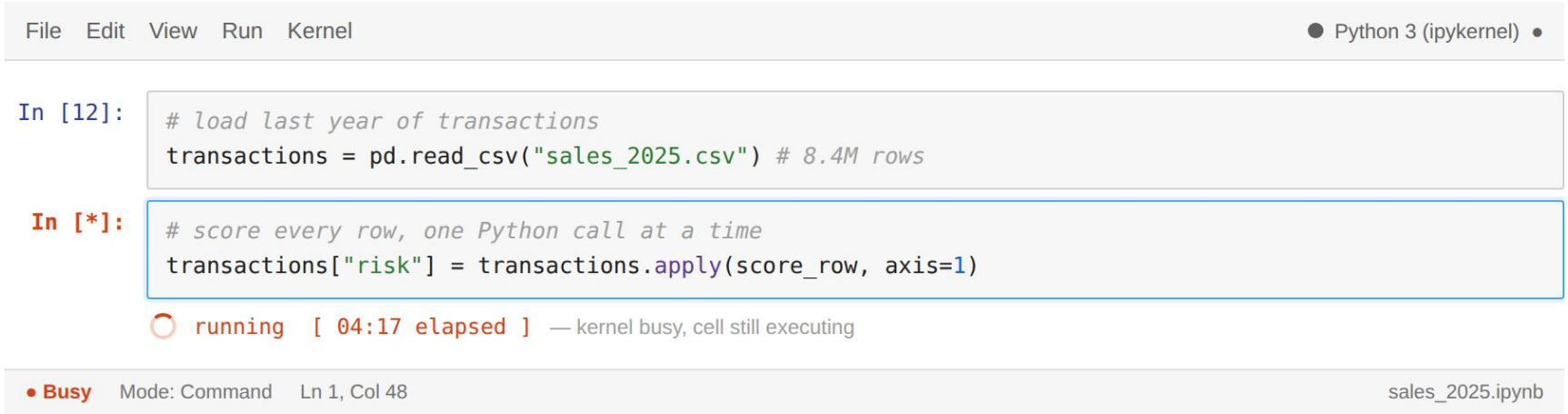
- Both topics (operating systems and parallel computing) allows for fairly different teaching approaches, across the range of fully theoretical and fully applied.
- Because of your study plan, I am planning for a mostly applied approach where I give you the necessary theoretical background to support hands on learning.
- I plan to focus on:
  - **Python**: THE programming language for data science
  - **Data science**: examples, problems, and exercises should be relevant for your background
  - **Docker container**: you have a variety of laptops, likely with Windows and macOS. We need an a learning environment that is equal for everyone, reproducible, and easy to fix.
  - **Linux operating system**: THE operating system for data science at scale and in production. Allows you to move beyond high-level frameworks (Jupyter notebooks and R studio), learning to use the command line and stand-alone applications.
- We are deciding how to setup this course together, depending on your interests, background, capabilities and so on. Only requirement: learning something new and useful for your future career. BTW, this applies to the exams too.

Why Should You Care?!

- Why should we care about operating systems and parallel computing?
- Typically taught in Computer Science and Software Engineering
- Can be very complex: a modern operating system requires more than 10M lines of code and is among the most complex software applications ever developed
- May require very strong programming skills: typically operating systems and parallel applications that scale well are written in C, C++, Rust, Go with advanced patterns
- May require a strong formal background: parallel and concurrent programming rest on a variety of formal methods and models.
- Our goals is focussing on what can be useful for a data scientist:
  - How do we efficiently and effectively manage large amount of data?
  - How do we efficiently and effectively scale our software simulations and models?
  - How do we think, measure, and interpret software performance?

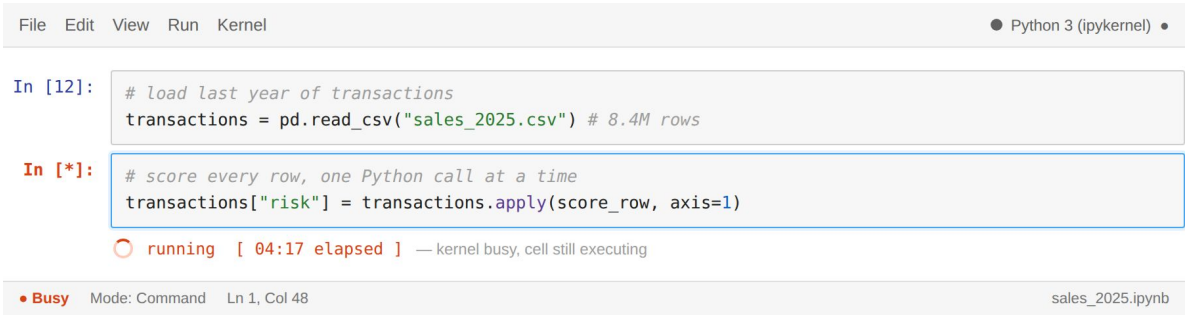
Scenario 1: The Slow Notebook

- Speed isn't about impatience, it's about how many iterations of thought we get per day, which is most of what separates good analysis from mediocre analysis.
  - We'll run fifty times an analysis we can run in 3 seconds, trying variations, catching mistakes, exploring.
  - We'll run twice an analysis that takes 40 minutes, and we'll ship the second version whether it's right or not.



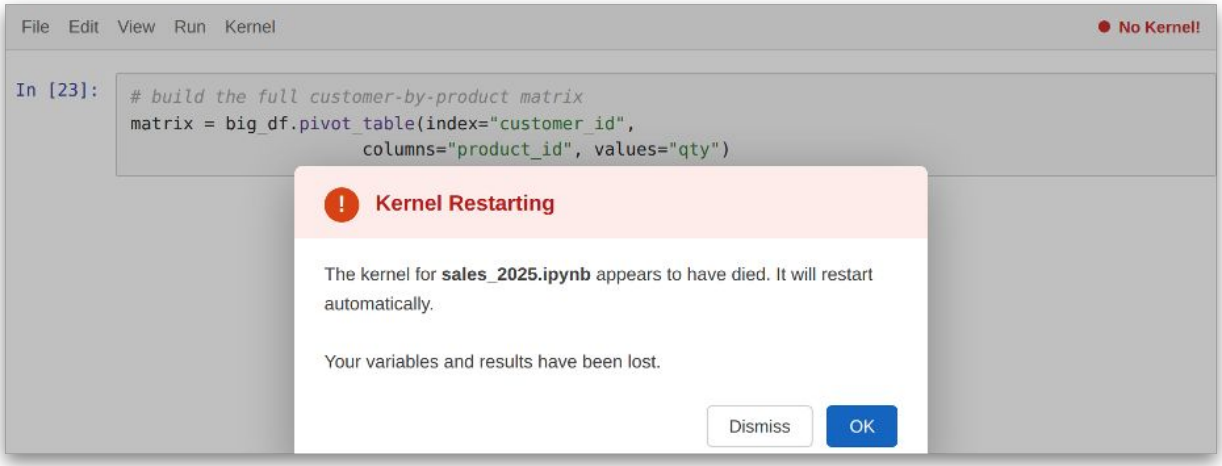
Scenario 1: The Slow Notebook

- Where did performance go?!
  - We are reading 8.4M rows from a CSV file into memory
  - Then `transactions["risk"] = transactions.apply(score_row, axis=1)` passes every raw to the function `score_row`. This is equivalent to a Python loop that calls a function 8.4M and store the result into a dedicated dataframe column.
  - A modern CPU does roughly 10^9–10^{10} simple operations per second, yet a plain Python loop over a few million rows can take minutes.
  - The hardware could have done this millions of times over, where did the time go?
  - We are been paying a 10-100x tax without knowing it existed.



Scenario 2: The Dead Kernel

- Why does this happen? Did we made a coding error? Is a notebook's bug? Do we need a new laptop?
- How do we diagnose it?
- The operating system deliberately killed our process to maintain other fundamental processes alive. See out-of-memory (OOM) killer, lecture 7.
- One of the most common way a real project stalls.



Scenario 3: The 100x Colleague

- Two analysts, same company, same laptop model, same dataset, same question.
- Barbara's (B) runs in 3 seconds; Adam's (A) in 5 minutes.
- B is not a better programmer. B knows things about the machine that A doesn't and every one of those things is on our syllabus.
- Fast analysts get high-stakes problems, because they can be trusted to deliver high quality on time. Speed compounds into responsibility and visibility.

Analyst A — same laptop, same data

In [4]:

```
%%time
totals = []
# loop over every one of 8.4M rows
for _, row in df.iterrows():
    totals.append(score(row))
df["score"] = totals
```

```
Wall time: 5min 12s
```

Analyst B — same laptop, same data

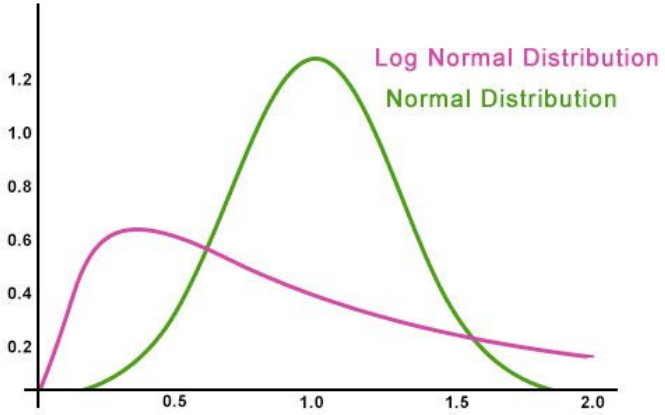
In [4]:

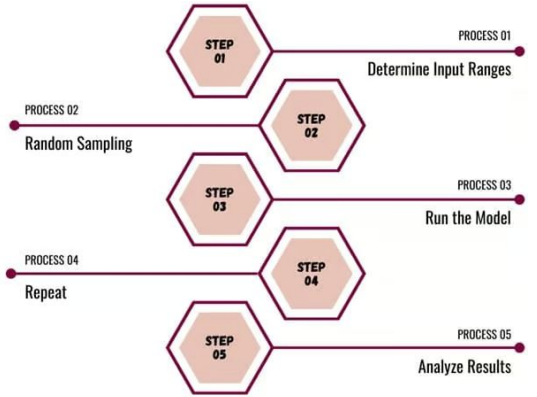
```
%%time
# the whole column at once
df["score"] = score_vectorized(df)
```

```
Wall time: 3.1s
```

Demo: Monte Carlo Simulation for Business

- Scenario:
  - We run a small business selling units of goods
  - Monthly demand is uncertain and can be modeled with a lognormal distribution.
  - Price per unit: €12
  - Variable cost per unit: €7.50
  - Fixed costs per month: €60,000
- We are hired as analyst consultants to answer this question:
  - What is the probability of a loss-making month?
- We use a Monte Carlo simulation:
  - A mathematical technique that uses random sampling, probability distributions, and repeated trials to forecast the possible outcomes of an uncertain event





01_three_ways.ipynb

- You do not need to understand every line today. You need to see the shape of the three approaches.
- The maths is in `mc_workers.py`; the notebook times three ways of running it.
- Comments explain each line in plain English.

Demo: Monte Carlo Simulation for Business

method	time	vs loop
Python loop	19.59s	1.0x
4 processes	10.44s	1.9x
NumPy	2.25s	8.7x

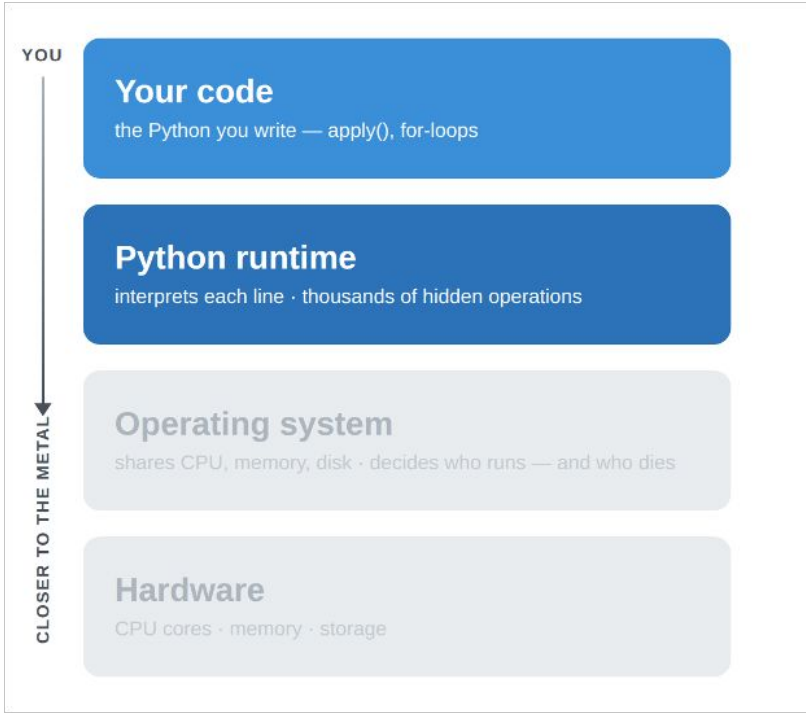
Fastest here: NumPy (8.7x the loop)

- *Example run, your numbers will differ*
- *Ordering should be the same given a large enough dataset*

- Correctness and performance are separate skills:
  - Every version of the code in the example is correct but only one goes to production.
  - We already know how to be correct when modeling and coding.
  - We need to learn what determines whether "correct" ever actually runs on real data.
- We are here to learn why each version runs at different speed, i.e., the mechanisms that allow code to run faster on an operating system and a runtime environment.
- Speed comes from types of parallelism, proper memory management, and knowledge of the available programming abstractions and runtime environment.

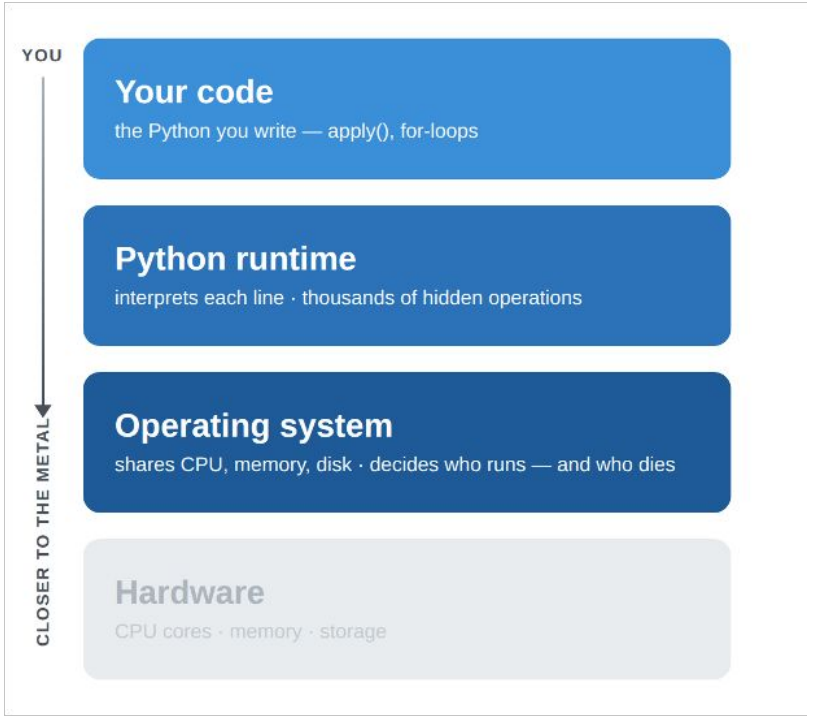
The Layers Under Our Notebooks

- Everything we experience as “slow” or “crashed” happens in one of four layers.
- **Layer 1 - Our code**: the Python we write, e.g. transactions.apply(score_row, axis=1).
- **Layer 2 - Python runtime:** it interprets each line of code and runs it. One line becomes thousands of hidden operations. E.g., dynamic typing, boxed objects, garbage collection, etc.
- Hint, scenario 1 lived here: the loop was slow because Layer 2 did heavy but invisible work on every row.
- This is the 10-100x tax from Scenario 1



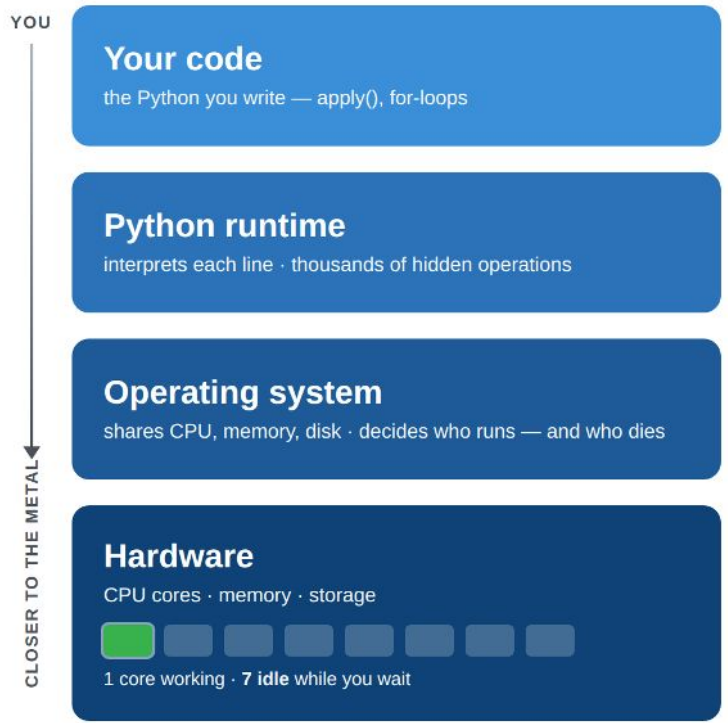
The Layers Under Our Notebooks

- **Layer 3 - Operating system (OS)**: manager between our program and the hardware.
- Every millisecond the OS decides what process gets the CPU, memory, disk read/write access
- OS also decides which process is killed.
- Hint, scenario #2 lived here: the OS killed our kernel to keep the machine alive.
- Production analytics run on a Unix machine driven from a terminal, with no desktop.
- Part I of this course = becoming fluent in this layer (sessions 2–13).



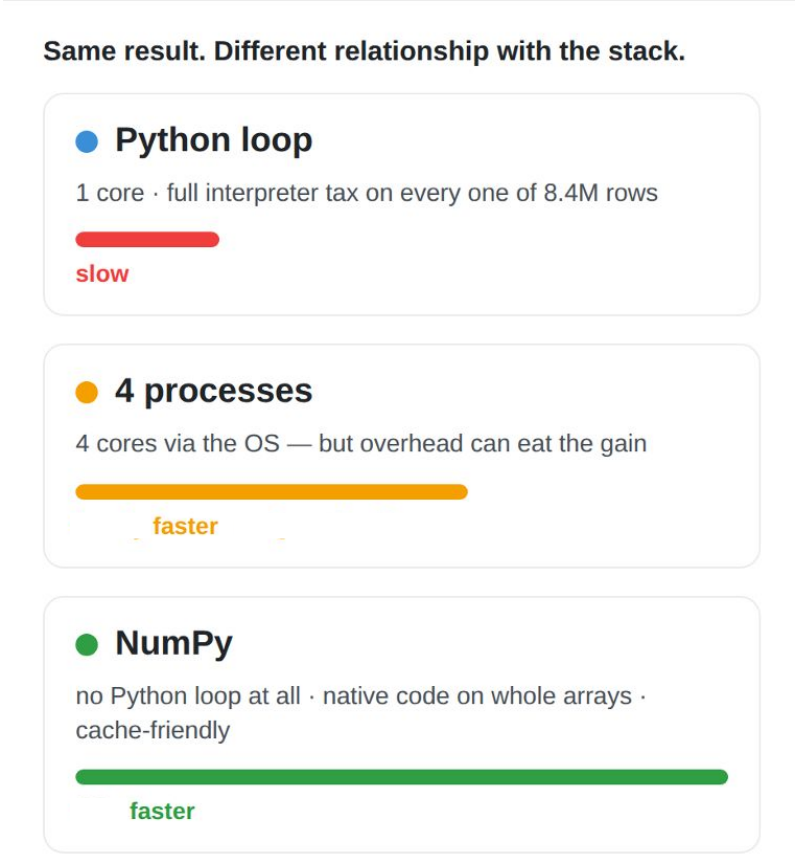
The Layers Under Our Notebooks

- **Layer 4 - The hardware**: the actual machine, e.g., CPU cores, memory, storage.
- Our laptop has ~8 cores. Our loop in the example code used one. The other seven sat idle while we waited.
- Note: idle capacity is not free! We have already paid for it.
- The same is true for clusters, clouds, GPUs, just at a larger scale. Same principles, bigger machine.
- Part II of this course = learning to use all available capacity on laptops and then on clusters and clouds (sessions 16–30).



Demo Performance Analysis

- Each version of the code has a different relationship with the four layers.
- Python loop: spend most of the time on a single core in layer 2.
- 4 processes: uses the OS (Layer 3) to get more cores (Layer 4), but overheads can eat almost 50% of the gain.
- NumPy: cooperates with every layer, vectorised, cache-friendly, offloaded to optimised native code.



A Lot of Questions, What About Some Answers?!

- **Part II:**
  - Why is the loop so slow? The cost of the runtime (Layer 2).
- **Sessions ~18-22:**
  - Why didn’t 4 workers give 4×? Overhead and Amdahl’s law.
- **Sessions ~19-20:**
  - How does NumPy win with no visible parallelism? Vectorisation and the memory hierarchy.
- **Sessions 7 & 23:**
  - What happens at 50 billion rows, not 80 million? When data outgrows memory.

Course Summary

Part I: The Machine Under Our Programs (Sessions 1–15)

1. Why the machine matters: course intro and the three-ways demo
2. Linux and the terminal I: navigation and the file system
3. Linux and the terminal II: pipes, scripting, and automation
4. Processes: the process abstraction, made visible
5. The kernel and system calls: the user/kernel boundary
6. Memory I: address spaces and virtual memory
7. Memory II: when our dataset doesn't fit RAM (swapping, thrashing, the OOM killer)
8. Storage, files, and formats: permissions and the storage hierarchy
9. OS Lab 1: process, memory, and pipeline experiments, **Lab Report 1**
10. Scheduling and the roots of concurrency: time-sharing and a first race condition
11. Security and cluster onboarding: permissions, SSH, and first cluster login
12. OS Lab 2: Automation on the cluster
13. Many processors: multicore, NUMA, and the bridge to Part II
14. Review and exam preparation
15. Partial exam covering sessions 1-14

Course Summary

Part II: Making It Fast (Sessions 16–30)

16. Profile first: measurement before optimization & **project launch**
17. Parallel architectures: shared vs. distributed memory, and the cluster
18. Threads, the GIL, and races: concurrency taught at the point of use
19. Multiprocessing: true CPU parallelism in Python
20. The analytics parallel ecosystem: joblib, NumPy/BLAS, and the oversubscription trap
21. Shared-memory lab: Parallelize and measure speedup, **Lab Report 2**
22. Performance and scalability: speedup, efficiency, Amdahl's law
23. Dask I: larger-than-memory computing
24. Dask II: distributed on the cluster, and MapReduce as a concept
25. Project studio 1: supervised implementation
26. GPUs for analytics: the throughput model and CuPy
27. Project studio 2: the measurement campaign
28. Group project presentations
29. Synthesis and frontier: beyond the course, plus exam prep
30. Final exam about sessions 16-29

TODOs

- Download the course container student guide from our Blackboard under supporting material/docker
- Follow the instructions to install Docker on your computer
- Follow the instructions to pull our course Docker container and run it on your laptop
- Use the Student Support Forum on our Blackboard if you have any issue. Note:

> *This forum is for students to share any issues they may have with the course container, notebooks, and coding. Students are invited to participate in the forum by sharing possible solutions and their own successful approaches to the posted issues. The professor will intervene in the discussions when needed, and participation in the forum (if any) will be part of the student's participation grade.*